

■ Fake News Detection System

AI Semester Project Report

Course	Artificial Intelligence
University	Sukkur IBA University
Department	Computer Science
Semester	Spring 2026
Group Members	Antra · Prerina · Ain-UI-Shuba

1. Task and Dataset

The goal of this project is to build a binary text classification system that determines whether a given news article is **real** or **fake**. Fake news — deliberately fabricated or misleading information — spreads rapidly through social media and causes measurable harm to public discourse. An automated detection system can assist readers in verifying content before sharing.

Dataset:

We used the publicly available **Fake and Real News Dataset** from Kaggle (Clément Bisailon, 2020). It contains two CSV files:

File	Label	Articles	Content
True.csv	Real (1)	~21,417	Reuters political news articles
Fake.csv	Fake (0)	~23,481	Fabricated/misleading news articles
Combined	Both	~44,898	Shuffled and split 80/20 train/test

2. Data Pre-processing

Raw news text cannot be fed directly into a machine learning model. We apply a multi-step pre-processing pipeline implemented in *utils.py*:

- **Concatenation:** The article title and body are joined into a single text field. Location tags such as "WASHINGTON (Reuters) —" are stripped via regex to prevent the model from learning source-formatting patterns.

- **Lowercase:** All text is converted to lowercase so that 'President' and 'president' are treated as the same token.
- **URL & Email Removal:** Regular expressions remove http/https URLs and email addresses.
- **HTML Tag Stripping:** Any residual HTML markup is removed.
- **Punctuation & Digit Removal:** Punctuation characters and standalone numbers are removed using Python's string.punctuation table.
- **Stopword Filtering:** A custom list of ~90 common English stopwords (no NLTK required) is applied. Tokens shorter than 3 characters are also dropped.

Code excerpt — preprocess_text() in utils.py:

```
text = text.lower()
text = re.sub(r'https?://\S+|www\.\S+', '', text)
text = text.translate(str.maketrans('', '', string.punctuation))
tokens = [w for w in text.split() if w not in STOPWORDS and len(w) > 2]
```

3. Architecture of the ML Pipeline

The system follows a classic supervised NLP pipeline. The diagram below illustrates each stage from raw input to final prediction:

Stage	Module	Description
1. Input	app.py / CLI	Raw news article text (title + body)
2. Pre-process	utils.py	Lowercase → strip URLs/HTML → remove punctuation → filter stopwords
3. Vectorise	TfidfVectorizer	Converts text to a sparse numerical matrix (10,000 TF-IDF features)
4. Classify	LogisticRegression or MultinomialNB	Outputs class label (0=Fake, 1=Real) and probability scores
5. Present	app.py (Streamlit)	Displays result card, confidence %, probability bars, history log

4. Training Objective, Model Selection & Design Choices

The task is **binary classification**. Given a feature vector x derived from TF-IDF, the model predicts $\hat{y} \in \{0, 1\}$. We trained and compared two algorithms:

a) Logistic Regression

Logistic Regression models the log-odds of a document belonging to the Real class as a linear combination of TF-IDF features. It is optimised by minimising the **binary cross-entropy (log-loss)** using the L-BFGS solver (max_iter=1000, C=1.0). It is interpretable — the magnitude of each weight shows how strongly a word predicts real or fake content, which we visualise in the top-features plot.

b) Naive Bayes (MultinomialNB)

Multinomial Naive Bayes applies Bayes' theorem under the independence assumption, estimating per-class word likelihoods. With Laplace smoothing (alpha=0.1) it is fast, memory-efficient, and performs well on count/frequency features such as TF-IDF. It serves as a strong and interpretable

baseline.

TF-IDF Design Choices

Parameter	Value	Reason
max_features	10,000	Limits vocabulary to the most discriminative words
ngram_range	(1, 2)	Captures single words AND two-word phrases (bigrams)
sublinear_tf	True	Applies $\log(1+tf)$ to reduce influence of very frequent words
Train/Test split	80% / 20%	Stratified split ensures balanced class ratios in both sets

5. Performance Evaluation

We evaluated both models using four complementary metrics on the held-out test set (20% of the data, ~8,980 articles):

Metric	Logistic Regression	Naive Bayes	What it measures
Accuracy	~98.0%	~94.2%	Overall correct predictions
Precision	~98.1%	~94.5%	Of predicted Real, how many are actually Real
Recall	~97.9%	~93.9%	Of all Real articles, how many were found
F1-Score	~98.0%	~94.2%	Harmonic mean of Precision and Recall
ROC AUC	~0.9993	~0.990	Area under the Receiver Operating Curve

The **confusion matrix**, **ROC curve**, **model comparison bar chart**, and **top-feature importance plot** are automatically saved to the *results/* folder when *train_model.py* is run. These visualisations are included in the repository.

6. Application of Techniques Taught in Class

- **Natural Language Processing (NLP):** Core topic of the AI curriculum. We applied tokenisation, stopwords removal, and the Bag-of-Words / TF-IDF representation, all covered in the NLP lecture week.
- **Supervised Learning:** Binary classification using labelled training data — a fundamental AI paradigm discussed across multiple lecture weeks. Labels are 0 (Fake) and 1 (Real).
- **Feature Engineering:** TF-IDF converts raw text into numerical features. The choice of max_features, n-gram range, and sublinear_tf demonstrates understanding of the feature-engineering process discussed in the model design lectures.
- **Model Selection & Comparison:** We trained two algorithms — Logistic Regression and Naive Bayes — and selected the best using held-out test accuracy and ROC AUC, reflecting the model-selection methodology taught in class.

- **Performance Evaluation Metrics:** Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and ROC AUC — all metrics introduced in the AI evaluation lectures — are computed and visualised.
- **Probabilistic Classification:** The Naive Bayes model applies Bayes' theorem, a core concept from the probability and uncertainty lecture. Logistic Regression outputs calibrated probability estimates.

7. Main Challenges Faced and How We Addressed Them

- **Source-formatting bias in the dataset:** Real news articles in True.csv began with location tags like 'WASHINGTON (Reuters) —', which the model could learn to recognise as a proxy for 'Real' rather than the actual content. We removed these patterns using a targeted regex in train_model.py before pre-processing, forcing the model to learn from the article content itself.
- **Dependency management:** Keeping the project deployable required minimising dependencies. We avoided NLTK (requires corpus downloads) and spaCy (large model), implementing our own stopwords list in utils.py. The entire system runs on five pip-installable packages.
- **Deploying model files to Streamlit Cloud:** Streamlit Cloud cannot run train_model.py during deployment (no dataset access). We documented the workflow clearly in README.md: train locally, commit the .pkl files to GitHub, then deploy. The app checks for missing files and shows a clear error.
- **Vocabulary generalisation:** Limiting features to 10,000 words with n-gram (1,2) and sublinear_tf=True improved generalisation over a simpler bag-of-words approach and helped the model capture common multi-word patterns (e.g., 'white house', 'fake news').

8. Tools & Technologies

Tool / Library	Version	Role
Python	3.9+	Core programming language
pandas	≥2.0	CSV loading, data manipulation
NumPy	≥1.24	Numerical operations & array handling
scikit-learn	≥1.4	TF-IDF, Logistic Regression, Naive Bayes, metrics
matplotlib	≥3.7	Evaluation plots (confusion matrix, ROC, bar chart)
Streamlit	≥1.32	Interactive web application UI
GitHub	—	Version control and deployment source
Streamlit Cloud	—	Free hosting / deployment platform

9. Conclusion

This project demonstrates an end-to-end AI system for fake news detection using classical NLP and machine learning techniques. A TF-IDF + Logistic Regression pipeline achieves approximately 98% accuracy on the Kaggle Fake and Real News dataset, outperforming the Naive Bayes baseline (~94%). The system is deployed as an interactive Streamlit web application, allowing any user to paste a news article and receive an instant classification with confidence scores.

The project applies multiple AI concepts covered in the course — supervised learning, NLP feature extraction, probabilistic modelling, model comparison, and performance evaluation — in a practical, real-world context. Future improvements could include deep learning models (LSTM, BERT) and a larger, more diverse multilingual dataset.

10. References

- [1] Bisailon, C. (2020). Fake and Real News Dataset. Kaggle.
<https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset>
- [2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- [3] Ramos, J. (2003). Using TF-IDF to Determine Word Relevance in Document Queries. Proceedings of the 1st Instructional Conference on Machine Learning.
- [4] Mitchell, T. (1997). Machine Learning. McGraw-Hill.
- [5] Streamlit Documentation. (2024). <https://docs.streamlit.io>